

ICS期末

Created @December 18, 2025 2:06 PM

Based on 往年题

Data

1. 请阅读以下代码

```
int main(){
    int x = 0;
    float y = 0.0;
    while((y - x < 1) && (y - x > -1)){
        x += 1;
        y += 1;
    }
    printf("%d %f\n", x, y);
}
```

问最终的输出为：

(提示 1：我们这里认为 int 型变量与 float 型变量运算会把 int 型转化为 float 型再进行运算)

(提示 2： $2^{24} = 16777216$)

(提示 3：本题中的舍入方式为向偶数舍入)

A. 16777218 16777217.000000

B. 16777218 16777216.000000

C. 16777217 16777216.000000

D. 16777217 16777215.000000

关于float舍入！

float表示的最大数为 2^{24}

float 能精确表示的最大“连续整数”是 2^{24} (16777216)

$2^{24}+1$ 如果向偶舍入，也会搞到 2^{24}

浮点数舍入：

一、浮点数会发生舍入的场景

浮点数（`float` / `double`）是有限精度的二进制近似表示（比如 `float` 仅能精确表示约 6~7 位有效数字，`double` 约 15~17 位），以下情况会触发舍入：

1. **数值本身无法精确表示**：比如 0.1、0.2、0.3 等小数，二进制无法用有限位数表示，存储时直接舍入为近似值（例如 `0.1f` 实际存储为 `≈0.10000000149`）。
2. **运算结果超出有效位数**：两个高精度数运算后，结果的有效位数超过浮点数的精度上限，会舍入（比如 `1.23456789f + 0.00000001f`，结果会舍入为 `1.2345679f`）。
3. **加减法的“对阶”操作**：浮点数加减前需要“对齐小数点（对阶）”，小阶数的数会被调整为大阶数的格式，低位精度会丢失并舍入（比如 `1e20f + 1f`，`1f` 会被舍入为 0，结果还是 `1e20f`）。
4. **溢出 / 下溢**：
 - 上溢：数值超过浮点数最大值，舍入为“无穷大”；
 - 下溢：数值小于浮点数最小值，舍入为 0。

二、会失效的数学算律（浮点数中不恒成立）

浮点数的舍入误差会导致部分数学算律失效，常见的是这 3 类：

1. 结合律

- 加法： $(a + b) + c \neq a + (b + c)$ 例子：`a=1e20f, b=1f, c=-1e20f`
 - `(a+b)+c = (1e20+1)-1e20 ≈ 0`
 - `a+(b+c) = 1e20+(1-1e20) ≈ 1`
- 乘法： $(a * b) * c \neq a * (b * c)$ （极端精度场景会出现）

2. 分配律

- $(a + b) * c \neq a * c + b * c$ 就是你之前问的 D 选项场景（比如 $fx=0.1f, fy=0.1f, fz=1e6f$ ，两边运算后因舍入误差相等）。

3. 消去律

- 若 $a * b = a * c$ ，不能推出 $b = c$ 例子： $a=1e-20f, b=1f, c=2f$
 - $a * b \approx 0$ ， $a * c \approx 0$ （两者舍入后相等），但 $b \neq c$

汇编：

立即数不能作为mov指令的dst

4. 现假定所有函数均使用 %rbp 寄存器描述栈帧，并且函数体进入时一定以 `pushq %rbp; movq %rsp, %rbp` 开始，函数体退出前一定以 `leaveq; retq` 结束。有编译器内置函数 `__builtin_return_address(1)` 表示获取调用当前函数的函数的返回地址，举例来说，若有如下代码：

```
long foo(void)
{
    return (long) __builtin_return_address(1);
}
```

同时函数 `bar()` 调用了函数 `foo()`，则函数 `foo()` 的返回值是函数 `bar()` 的返回地址（即 `bar()` 调用完成后控制流转向的地址），保证 `bar()` 和 `foo()` 都不是内联函数，则将函数 `foo()` 编译，生成的汇编指令可能是（）

4. 答案：D。在 `foo()` 中读取 `0(%rbp)` 可以获得 `foo()` 开头保存的 `%rbp`，也就是外层函数 `bar()` 的栈帧地址，向上偏移 8 即可获得函数 `bar()` 的返回地址。

2. 下列关于处理器体系结构的说法，**正确**的是：

- A. RISC 的设计理念是精简指令，为了减少指令的数量，它使用的指令长度可变。
- B. 在设计处理器时，硬件上复制逻辑块的成本要比软件中复制代码的成本低得多，而且在硬件系统中处理各种特殊情况也比用软件处理简单。
- C. 在处理器的流水线中，异常指令在到达写回阶段之前，不用让异常事件影响流水线中的指令流，只需要禁止后面的指令更新条件码和修改内存等产生永久影响的操作即可。
- D. 在处理器的流水线中，在译码阶段需要处理转发 (Forwarding)。如果有相同目标的多个转发源，赋予不同优先级是十分重要的，来自越靠后流水线阶段的转发源的优先级越高。

答案：C。

- A. CISC 使用可变长度指令，RISC 使用的是固定长度的指令。
- B. 我们希望复用各类算术/逻辑单元恰恰是因为硬件上复制逻辑块的成本更大。

D越靠前，优先级越高

存储Cache

5.答案：C

解析：A 选项，RISC 指令集有复杂化的趋势，早期 RISC 通常指令数量小于 100 条；B，CISC 的指令是不定长的，有些长度比 RISC 短；C 正确；D，CISC 一定会将返回地址压入栈中，不可能避免内存引用。

本题考查对两种指令集的理解。

9.以下关于存储设备与存储器层次结构的说法，正确的是：

- A.一个有 5 个盘片，每个面 10000 条磁道，每条磁道平均有 400 个扇区，每个扇区有 512 个字节的硬盘容量为 20.48GB。
- B.SRAM 的存取速度快，但是断电后数据会丢失，DRAM 的存取速度较慢，但断电后数据不会丢失。
- C.在存储器层次结构中，上一层存储的信息一定是下一层存储的信息的一个子集。
- D.基于缓存的存储器层次结构只利用了程序的时间局部性，没有利用程序的空间局部性，因为缓存不会存储我们没访问过的元素。

选项 A：硬盘容量计算（正确）

硬盘容量的计算公式是：容量=磁头数（盘面数）×磁道数/盘面×扇区数/磁道×扇区大小

题目参数：

- 盘片数 = 5 → 盘面数（磁头数）=5×2=10（每个盘片有上下 2 个面）；
- 每个面磁道数 = 10000；
- 每条磁道扇区数 = 400；
- 每个扇区大小 = 512 字节。

计算过程： $10 \times 10000 \times 400 \times 512 = 20480000000$ 字节

注意：题目中“GB”采用十进制换算（1GB=10⁹字节），因此： 20480000000 字节 = 20.48×10^9 字节 = 20.48GB

因此选项 A 的描述是正确的。

选项 B：SRAM 与 DRAM 的特性（错误）

SRAM（静态随机存储器）和 DRAM（动态随机存储器）的核心特点：

- SRAM：用触发器存储数据，速度快、功耗高，**断电后数据丢失**；
- DRAM：用电容存储数据，需周期性刷新，速度慢、功耗低，**断电后电容放电，数据也会丢失**。

选项 B 中“DRAM 断电后数据不会丢失”的描述错误。

选项 C：存储器层次结构的子集关系（错误）

存储器层次结构（如寄存器→Cache→内存→硬盘）中，上一层（如 Cache）是下一层（如内存）的 **“缓存副本”**，但并非 **“严格的子集”**：

- 当数据被修改后（如 Cache 中的“脏数据”还未写回内存），Cache 中的内容与内存不一致，此时 Cache 不是内存的子集。

因此“上一层一定是下一层的子集”的描述错误。

选项 D：缓存的局部性利用（错误）

缓存（如 Cache）同时利用了**时间局部性**和**空间局部性**：

- 时间局部性：最近访问的内容会被再次访问（缓存保留常用数据）；
- 空间局部性：访问一个数据时，其周围数据也会被访问（缓存会加载“数据块”，如 Cache Line）。

选项 D 中“没有利用空间局部性”的描述错误。

4. 下列关于存储设备的说法，**正确**的是：

- A. 随机访问存储器（Random-Access Memory, RAM）分为静态的和动态的，这两类也被统称为 SDRAM。
- B. 只读存储器（Read-Only Memory, ROM）只能被编程一次，也就是它们只能被写一次。只读存储器也被称为非易失性存储器。
- C. 固态硬盘（Solid State Disk, SSD）的随机写比读慢，因为擦除块需要较长的时间，如果写操作试图修改一个包含已有数据的页，那么这个块中所有带有有用数据的页都要被复制到一个新块中。
- D. 组相联高速缓存很容易产生“抖动”问题，因此我们引入了直接映射高速缓存。

答案：C。

A. SDRAM 是同步 DRAM，而不是 SRAM 与 DRAM 的统称。

B. 只读存储器是一个历史遗留的称呼，并非所有 ROM 都只能被写一次。ROM 和 NVM 也不是相同的概念。

C. 正确，参见中文版课本 P415。

D. 相比组相联高速缓存，直接映射高速缓存更容易发生抖动问题。

程序优化

C.大多数编译器会改变浮点数的合并运算（如加法和乘法）顺序以提高程序性能。
错，因为浮点数没结合律，大多不优化

链接Linkage

（Linux 工具链）简单，基础 1-2 min

1. 以下关于 Linux 系统上处理可执行文件的工具的说法不正确的是
 - A. 使用 `objdump` 反汇编 `.text` 节的机器码
 - B. 使用 `readelf` 读取文件的节头部头 (section header) 和程序头部表 (program header)
 - C. 使用 `ls` 查询文件是文本文件还是二进制文件
 - D. 使用 `gdb` 加载可执行文件、设断点，然后单步调试运行

C 错误

解析：

- A 正确。`objdump -dj .text [file]`
- B 正确。节头部表：`readelf -S [file]` 程序头部表：`readelf -l [file]`
- C 错误。`ls` 只是取得文件的元数据，这与文件内容无关，而 linux 文件元数据中也不包含对 binary 或者 text 编码属性的描述。其他诸如 `grep` 和 `file` 的工具使用 heuristics 确定文件的类型。
- D 正确。`gdb` 可以支持单步调试。

1. 针对如下代码，假定编译器不做任何编译优化，同时编译系统在预处理a.c和b.c 时可以顺利找到common.h。运行gcc a.c b.c命令，发现报错。已知报错信息中含有如下内容：

```
4
collect2: error: ld returned 1 exit status
// a.c
#include <stdio.h>
#include "common.h"
int main() {
```

```
scanf("%d\n", &x);
funcB();
printf("%d\n", x);
}
// b.c
#include "common.h"
int funcB(void) {
    x++;
}
// common.h
int x = 1;
int funcB(void);
```

则下列说法正确的是：

- A. 在编译系统及二进制工具链对本题所涉源代码进行处理过程中，会产生名为a.o、b.o的中间临时文件，并将其链接到一起。
- B. 根据题目中的报错信息描述进行推断，报错信息是由编译系统及二进制工具链中的预处理器输出的。
- C. 报错原因是a.c、b.c、common.h其中之一含有语法错误。这个错误可以通过调整编译顺序来解决，即gcc b.c a.c。
- D. 在编译系统及二进制工具链对本题所涉源代码进行处理过程中，所产生的中间目标文件的.text节中，对于变量x的引用都需要在链接时重定位。

答案：D

本题考察编译和链接的相关知识。

A. 用该命令编译时，common.h只被预处理器复制到a.c与b.c中，而不参与之后的编译阶段。产生的中间临时文件名是随机的，并非一定为a.o和b.o。

B. 报错原因是int x=1; 会被预处理器**分别复制到a.c与b.c中，从而导致强符号a重定义，链接时报错，类似**

```
/usr/bin/ld:
/tmp/cckR3OM5.o:(.data+0x0):
multiple
definition of `a'; /tmp/ccSDjtQc.o:(.data+0x0): first
defined here
collect2: error: ld returned 1 exit status
```

- C. 各种信息都说明这是链接器报错；最明显的是：ld是GNU 链接器的简称。
- D. 确实，任何在编译时无法确定地址的数据引用需要重定位

含有定义的函数即使没有被使用也会存放在符号表中，可供其他.o文件使用。

初值非0的局部静态变量被存放在.data节。后半句包含解释。

可以使用objdump -t main.o 验证答案：

ECF

1、关于进程和异常控制流，以下说法正确的是：

- A、调用 `waitpid (-1, NULL, WNOHANG & WUNTRACED)` 会立即返回：如果调用进程的所有子进程都没有被停止或终止，则返回 0；如果有停止或终止的子进程，则返回其中一个的 ID。
- B、进程可以通过使用 `signal` 函数修改和信号相关联的默认行为，唯一的例外是 `SIGKILL`，它的默认行为是不能修改的。
- C、从内核态转换到用户态有多种方法，例如设置程序状态字；从用户态转换到内核态的唯一途径是通过中断/异常/陷入机制。
- D、中断一定是异步发生的，陷阱可能是同步发生的，也可能是异步发生的。

答案：C

简单

A中option参数应该使用 `|` 运算结合。B中SIGKILL不是唯一的例外，例外共有两个：SIGKILL、SIGSTOP。D陷阱一定是同步发生的

进程只在内核态切换到用户态时检测并处理信号

7. 下列陈述中，**错误**陈述的数量为：

- ①进程调度的实现在一定程度上依赖于定时器中断 (timer interrupts)。
- ②execv() 函数成功调用后不会返回。
- ③使用 fork() 创建的子进程与父进程对内存的修改是共享的。
- ④在 X86-64 Linux 中，访存缺页异常属于故障。
- ⑤异常处理的返回行为包括：返回当前指令、返回下一条指令和终止 (abort)。
- ⑥如果处理得当（如设置 SIGFPE 的信号处理函数），除以零引发的除法错误不会导致终止 (abort)。
- ⑦单核处理器上，可以通过不断地上下文切换交替运行并发的程序，给用户呈现一种并发的程序在同时运行的感觉。
- ⑧进入信号处理函数时会创建新的进程。

A. 0 个 B. 1 个 C. 2 个 D. 3 个

答案：C， ③⑧错误。使用 fork 创建的子进程和父进程在 初始阶段 的内存内容是相同的，但它们并不是直接共享内存。父子进程初期的内存内容是相同的，但它们对内存的修改是 独立的，并不共享。进入信号处理函数并不会创建新的进程。

10

1. 下列关于系统 I/O 的说法中，正确的是（）：
- A. Linux shell 创建的每个进程开始时都有三个打开的文件：标准输入（描述符为 0），标准输出（描述符为 1），标准错误（描述符为 2），这使得程序始终不能使用保留的描述符 0,1,2 读写其他文件。
 - B. Unix I/O 的 read/write 函数是异步信号安全的，故可以在信号处理函数中使用。
 - C. RIO 函数包的健壮性保证了对于同一个文件描述符，任意顺序调用 RIO 包中的任意函数不会造成问题。
 - D. 使用 `int fd1 = open("ICS.txt", O_RDWR);` 打开 ICS.txt 文件后，再用 `int fd2 = open("ICS.txt", O_RDWR);` 再次打开文件，会使得 fd1 对应的打开文件表中的引用计数 `refcnt` 加一。

答案：B（简单）

- A. 描述符 0, 1, 2 可以重定向到其他文件。
- B. 正确，教材 P534。
- C. 有缓冲区和无缓冲区的不可以交叉使用，教材 P629。
- D. 多次打开文件应该对应着多个打开文件表，教材 P635。

机制	结论
<code>O_TRUNC</code>	清空文件， <code>offset=0</code>
每次 open	得到 独立 file offset
fd1 (<code>O_APPEND</code>)	每次 write 都跳到文件末尾
fd2	普通写，从自己的 <code>offset</code> 走
fd	一直复用， <code>offset</code> 累积

VM

1. 下列关于虚存和缓存的说法中，正确的是：

- A. TLB 是基于物理地址索引的高速缓存
- B. 多数系统中，SRAM 高速缓存基于虚拟地址索引
- C. 在进行线程切换后，TLB 条目绝大部分会失效
- D. 多数系统中，在进行进程切换后，SRAM 高速缓存中的内容不会失效

答案：D

解析：属于中等题。考察虚拟内存和高速缓存的关系，并且和进程/线程有一定联系。

A 课本原话：A TLB is a small, virtually addressed cache where each line holds a block consisting of a single PTE.

B 课本原话：Although a detailed discussion of the trade-offs is beyond our scope here, most systems opt for physical addressing.

C 线程共享虚拟地址空间，所以 TLB 中条目不会失效

D 课本原话：With physical addressing, it is straightforward for multiple processes to have blocks in the cache at the same time and to share blocks from the same virtual pages.

答案：D

A. 在 Linux 中，fork() 系统调用会创建一个子进程。为了实现高效的进程复制，内核使用写时复制 (COW) 技术，在 fork() 后，父进程和子进程共享相同的物理页面，直到其中一个进程尝试写入这些页面时，才会为该进程分配独立的物理页面。为了确保写时复制机制的正确性，这些页面会被标记为只读，只有在写操作发生时才会进行复制。

B. execve() 函数加载一个新的程序并替换当前进程的映像，包括程序计数器 (PC) 和其他执行上下文。程序计数器会被设置为新程序的起始地址。

C. munmap() 函数用于解除内存映射区域的映射，删除该区域后，再尝试访问该内存区域会导致访问非法内存，从而引发段错误 (Segmentation Fault)。

D. 错误。交换空间 (swap space) 用于在物理内存不足时存储不常用的页面。交换空间的大小限制了操作系统能够为进程分配的虚拟内存的总量，尤其是在内存不足的情况下。如果系统没有足够的物理内存来满足所有进程的虚拟内存需求，交换空间的大小就成为一个限制因素。因此，交换空间与当前运行进程能够分配的虚拟页面总数是有关系的。

一、COW 机制到底是什么？

COW 是操作系统的内存优化技术，核心是“能共享就先共享，要修改再复制”，目的是节省内存、提高效率。

核心原理

1. **初始共享**：当两个实体（比如父进程 & 子进程、内存映射的进程）需要用同一块内存时，操作系统不直接复制内存，而是让它们**共享同一块物理内存**，同时把这块内存标记为「只读」。
2. **写操作触发复制**：当其中一个实体要**修改**这块共享内存时，操作系统才会真正复制一份新的物理内存给这个实体，让它在自己的副本上修改——这样另一个实体的内存不会被影响。
3. **适用场景**：常见于 `fork`（创建子进程）、`mmap`（内存映射）的 `MAP_PRIVATE` 模式等场景。

二、结合题目，看 COW 怎么发挥作用

先理题目里的核心流程：父进程用 `mmap` 映射文件 → `fork` 子进程 → 父子进程分别修改映射内存 → 父进程重新映射文件。

我们一步步看 COW 在其中的作用：

步骤 1：父进程首次 `mmap`（触发 `MAP_PRIVATE` 的 COW）

父进程执行：

c

运行

```
char* bufp = Mmap(NULL, 2, PROT_READ | PROT_WRITE, MAP_PRIVATE, fd, 0);
```

- `MAP_PRIVATE` 是「私有映射」：意味着父进程对这块内存的修改**不会写回原文件**，而是存在自己的内存副本里（这本身就是 COW 逻辑）。
- 此时 `bufp` 指向的内存内容是文件 `input.txt` 的原始内容：`"12"`。

步骤 2：`fork` 子进程（触发父子进程的 COW 共享）

父进程执行 `Fork()` 后：

- 子进程不会复制父进程的整个地址空间，而是和父进程**COW 共享** `bufp` 指向的物理内存（内容还是 `"12"`）。

- 操作系统会把这块共享内存标记为「只读」—— 为后续的 COW 复制做准备。

步骤 3：子进程修改内存（触发 COW 复制）

子进程执行：

c

运行

- `bufp = '2'; // 修改第一个字节`
- 子进程要修改「只读的共享内存」，触发 COW 机制：操作系统会为子进程**复制一份独立的内存副本**。
- 子进程在自己的副本里，把第一个字节改成 `'2'`，所以子进程的 `bufp` 内容变成 `"22"`。
- 子进程执行 `Write(1, bufp, 2)`，输出 `"22"`。

步骤 4：父进程修改内存（再次触发 COW 复制）

父进程等子进程结束后，执行：

c

运行

- `(bufp+1) = '1'; // 修改第二个字节`
- 父进程要修改「只读的共享内存」，同样触发 COW：操作系统为父进程复制一份独立的内存副本。
- 父进程在自己的副本里，把第二个字节改成 `'1'`，所以父进程的 `bufp` 内容从 `"12"` 变成 `"11"`。
- 父进程执行 `Write(1, bufp, 2)`，输出 `"11"`。

步骤 5：父进程重新 `mmap`（无 COW，读原始文件）

父进程重新执行：

c

运行

```
bufp = Mmap(NULL, 2, PROT_READ, MAP_PRIVATE, fd, 0);
```

- 这次 `mmap` 是重新映射文件 `input.txt` 的前 2 字节，而之前的修改是「私有映射」（不会写回文件），所以新 `bufp` 的内容是文件原始内容 `"12"`。
- 父进程执行 `Write(1, bufp, 2)`，输出 `"12"`。

三、最终输出与 COW 的关系

把各步骤的输出连起来：子进程输出 `"22"` + 父进程第一次输出 `"11"` + 父进程第二次输出 `"12"` → 总输出是 `"221112"`（对应选项 A）。

这里的关键就是 **COW** 让父子进程修改内存时各自复制副本，所以子进程改 `"12"` 为 `"22"`、父进程改 `"12"` 为 `"11"`，互不影响——这都是 COW 机制在起作用。

Network

4. 下列关于全球 IP 因特网的说法中，正确的是
- TCP/IP 协议规定网络字节顺序取决于不同主机的字节顺序，从而方便主机和网络之间收发数据
 - 域名集合和 IP 地址集合的映射关系目前由 http 来维护
 - 多个域名可以映射到同一个 IP 地址，但一个域名不可以映射到多个 IP 地址
 - 从内核的角度来看，套接字是通信的一个端点；从用户程序的角度来看，套接字是一个有相应描述符的打开文件

答案：D

解析：

- 网络字节顺序是大端法
- DNS 负责维护域名集合和 IP 地址集合的映射关系
- 一个域名也可以被映射到多个 IP 地址

5. 下列关于网络的说法中，错误的是
- 在 TCP 连接中，客户端和服务端都是指主机
 - 网络按照覆盖范围的大小可以分成局域网和广域网，其中一种流行的局域网技术是以太网
 - 网络可以被主机认为是一种 I/O 设备，是数据源和数据接收方
 - 路由器可以把多个不兼容的网络连接起来组成一个互联网络

答案：A

解析：简单题，考察 11.1 部分内容

- 在该模型中，客户端和服务端指进程
- BCD 均正确

A. IP 协议提供的是从主机到主机的通信机制，而不是从进程到进程。IP 协议主要负责在网络上按数据包的形式传递数据，提供的是网络层的服务。进程到进程的通信是由传输层的协议（如 TCP 和 UDP）来提供的。

并发